GRAF SAINS DATA

Nama : Moh Annas Shobari
Kelas : 2023E
NIM : 23031554241

I. Pertanyaan Pendahuluan

a. neo4j
b. Beberapa akses lain yang bisa digunakan untuk mengakses database lain dalam 1 instance
    1. DDL
        i. CREATE DATABASE nama_db
        ii. SHOW DATABASES
        iii. DROP DATABASE nama_db
    2. DML
        i. CREATE
        ii. MATCH
        iii. MERGE
        iv. SET
    3. Python
c. Graf Berarah dengan relasi bertipe KNOWS

II. Soal UTS

1. Periode waktu yang digunakan
   i. P1 → 3 Juni – 22 Agustus 2015
   ii. P2 → 23 Agustus – 12 September 2015
2. File telah disimpan dengan nama
   i. events_P1.csv
   ii. events_P2.csv
3. Komposisi Subgraph Menggunakan Python API
   Perhitungan kompisisi subgraph dilakukan menggunakan Cypher pada instace Neo4j GrafSada, yang menyimpan graf berdasarkan keseluruhan data events.csv, Query dijalankan melalui python API untuk memperoleh jumlah simpul berdasarkan jenis labelnya.
   Hasil perhitungan menunjukkan bahwa graf memiliki **1.407.580** simpul berlabel **Visitor** dan **235.061** simpul berlabel **item**, sehingga totalnya adalah **1.642.641 simpul**. Simpul Visitor merepresentasikan pengunjung yang melakukan interaksi, sedangkan simpul item merepresentasikan produk yang menjadi objek interaksi.
   Cypher menggunakan klausa **MATCH** untuk mengambil simpul, **UNWIND** untuk memisahkan label, dan **count(DISTINCT n)** untuk menghitung jumlah simpul unik pada setiap jenis label. Hasil kemudian diurutkan menggunakan **ORDER BY**. Dengan demikian, query tersebut memberikan gambaran mengenai komposisi simpul pada graf yang tersimpan di instance grafsada.

	jenis_simpul	jumlah_simpul
0	Visitor	1407580
1	item	235061

4. VSM user (Visitor) dibangun untuk merepresentasikan setiap Visitor berdasarkan pola interaksinya terhadap item pada dataset RetailRocket periode P1. **Matriks** yang dihasilkan memiliki shape sebesar (**861498, 189522)**, yang menunjukkan **861.498** baris visitor dan **189.522** kolom item. Setiap elemen matriks bernilai biner, yaitu **1** apabila Visitor **pernah**

```
=== VSM USER ===
Shape: (861498, 189511)
Jumlah pasangan interaksi unik: 1291947
```

   berinteraksi dengan item dan **0** apabila **tidak terdapat interaksi**. Matriks memiliki **1.291.946** nilai **nonzero** dengan **density** sebesar **0,00079133%,** sehingga tergolong **sangat sparse** (tidak padat). Selanjutnya, representasi ini dapat digunakan untuk menghitung kemiripan antar-visitor dan membentu graf proyeksi menggunakan NetworkX.
5. VSM item dibangun dengan mentransposisi matriks interaksi VSM user sehingga setiap baris merepresentasikan item dan setiap kolom merepresentasikan Visitor. Matriks ini memiliki shape sebesar (**189522**, **861498**), yang menunjukkan **189.522** baris item dan **861.498** kolom visitor. Jumlah **nonzero** tetap sebesar **1.291.946** dan **density** tetap **0,00079133%**, karena proses transpose hanya menuka orientasi matriks. Representasi ini digunakan untuk membandingkan pola pengunjung antar-item dan membentuk graf proyeksi kemiripan item menggunakan NetworkX

```
=== VSM ITEM ===
Shape: (189511, 861498)
Jumlah pasangan interaksi unik: 1291947
```

6. Berdasarkan hasil eksplorasi VSM pada nomor 4 dan 5, **proyeksi item-item** berpotensi **lebih efisien** dibandingkan proyeksi User-user untuk analisis kemiripan dengan pendekatan perbandingan pasangan simpul. Hal ini adalah karena jumlah simpul item sebanyak **189.522**, sedangkan jumlah simpul visitor mencapai **861.498.** sehingga, jika dihitung secara matematis, jumlah pasangan item yang mungkin dibandingkan sekitar **18,0 miliar**, sedangkan jumlah pasangan visitor/user mencapai sekitar **371,1 miliar**.
7. Perhitungan Similarity
   a. 5 Records dengan similarity tertinggi

```
MATCH (source:item {itemid: 5411})
      -[r:SIMILAR_TO]->
      (target:item)

RETURN
    source.itemid AS item_acuan,
    target.itemid AS item_mirip,
    r.similarity AS similarity

ORDER BY r.similarity DESC
LIMIT 5
```

   b. Menampilkan nilai similarity

```
neo4j$ MATCH (source:item)-[r:SIMILAR_TO]->(target:item) WHERE
```

Table | Raw

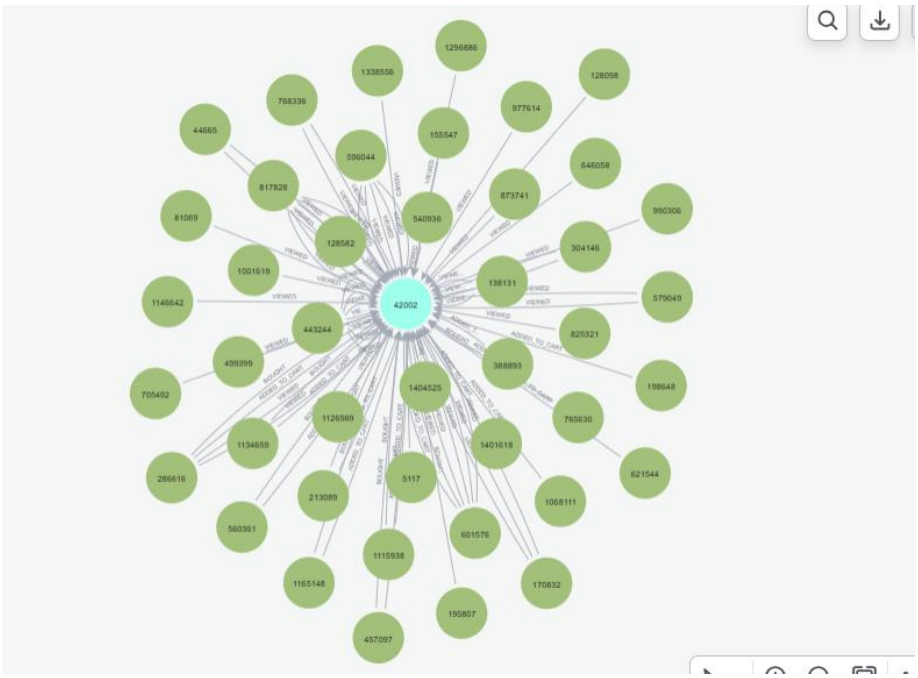
	item_acuan	item_mirip	similarity	metode	periode
1	5411	33030	0.15045937205113746	"cosine_similarity"	"P1"
2	5411	33956	0.08524826841213412	"cosine_similarity"	"P1"
3	5411	37807	0.08454968154078789	"cosine_similarity"	"P1"
4	5411	358234	0.08139101201768408	"cosine_similarity"	"P1"
5	5411	375492	0.06515903809680011	"cosine_similarity"	"P1"

8. Kondisi sebelum dan sesudah perhitungan similarity

Cypher Before :

```
MATCH (v:Visitor)-[r]->(i:item)
WHERE type(r) IN ['VIEWED', 'ADDED_TO_CART', 'BOUGHT']
RETURN v, r, i
LIMIT 100;
```

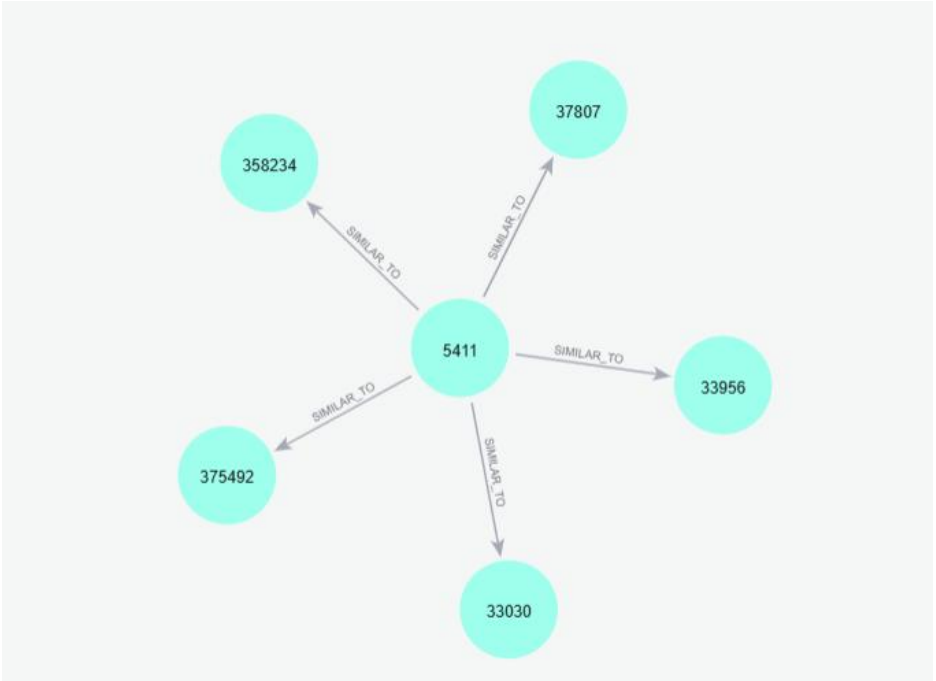
Result:



Cypher After :

```
MATCH (source:item)-[r:SIMILAR_TO]->(target:item)
WHERE r.period = 'P1'
RETURN source, r, target
ORDER BY r.similarity DESC
LIMIT 5;
```

Result :



III. Analytics

1. Pilih seorang user secara acak (tentukan proses acak-nya, dan harus bisa di-reproduksi), lalu
   a. tuliskan skrip Python dalam merekomendasikan KNN-nya!

```
K = 5

if n_users < K + 1:
    raise ValueError("Jumlah user terlalu sedikit untuk memilih 5
tetangga.")

target_idx = user_to_idx[target_user]
target_vector = user_item[target_idx]

# Cosine similarity sparse terhadap seluruh user; hanya satu baris
hasil
similarities = cosine_similarity(target_vector, user_item).ravel()

similarity_df = pd.DataFrame({
    "visitorid": user_encoder.classes_,
    "similarity": similarities
})

# Hilangkan target user sendiri
similarity_df = similarity_df[
    similarity_df["visitorid"] != target_user
]

knn = similarity_df.sort_values(
    ["similarity", "visitorid"],
    ascending=[False, True]
).head(K).reset_index(drop=True)

print("User target:", target_user)
print(f"5 tetangga terdekat (K={K}):")
display(knn)
```

Output :

```
User target: 1088457
5 tetangga terdekat (K=5):
```

	visitorid	similarity
0	115947	0.707107
1	744148	0.707107
2	1085304	0.707107
3	1176088	0.707107
4	1191780	0.707107

b. Berikan 3 Item yang diiklankan untuk k = 5 pada penentuan KNN-nya!

Top-3 item rekomendasi:

	itemid	score
0	39449	2.828427
1	419060	0.707107
2	3	0.000000

2. Berdasarkan hasil rekomendasi Anda no. 1b Analytics, hitunglah:

a. Hit@5

```
Indikator relevansi pada top-5: [0, 0, 0, 0, 0]
Hit@5: 0
```

b. NDCG@5

```
Relevansi top-5: [0, 0, 0, 0, 0]
DCG@5: 0.0
IDCG@5: 1.0
NDCG@5: 0.0
```